MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2025/2026

EXPERIMENT 3 :

DESIGNING A MOTION ON-ACTIVATED RFID SYSTEM

SECTION 2

GROUP 12

NAME	METRIC NO.
AHMAD FARHAN BIN AHMAD NAHRUDI	2317373
MUHAMMAD AIMAN BIN SAFAWI	2318249
MOHD NAFIZ ZUHAIREE BIN MOHD NEEZAR	2317423

DATE OF SUBMITION :
5/11/2025

ABSTRACT

This experiment focuses on exploring how serial communication works between an Arduino microcontroller and two external devices: an Inertial Measurement Unit (IMU) and a Radio Frequency Identification (RFID) module. The main goal is to understand how data is transmitted and received through the serial interface while ensuring smooth and accurate communication between multiple sensors. The IMU module collects motion data such as acceleration and orientation, whereas the RFID module reads and identifies tag information. By connecting both modules to the Arduino, real-time data can be transmitted to a computer and displayed using the Arduino Serial Monitor. The experiment demonstrates how these devices can work together without causing data interference, highlighting the importance of proper serial configuration, such as setting the correct baud rate and managing data buffers. Overall, the experiment successfully shows how serial communication enables efficient interaction between sensors and a microcontroller, forming the foundation for many automation and embedded system applications.

TABLES OF CONTENTS

TABLES OF CONTENTS

AB1STRACT..... 2
TABLES OF CONTENTS.....3
TASK 1..... 5
INTRODUCTION.....5
MATERIAL AND EQUIPMENT.....6
EXPERIMENTAL SETUP.....7

Circuit setup..... 6
Circuit Assembly.....6
Overview of Procedure.....7
Hardware Configuration Procedure.....8
Arduino Programming Procedure..... 9
Python Data Acquisition and Real-Time Plotting Procedure.....10
Improvements Made to the Starter Code.....12
RESULTS..... 13
Visual Output.....13
TASK 2.....14
MATERIALS AND EQUIPMENT.....14
EXPERIMENTAL SETUP.....15
Hardware Connections..... 15
System Flow.....15
METHODOLOGY.....16
System Preparation.....17
Arduino Programming.....19
Python Programming.....23
DATA COLLECTION..... RFID
UID Samples..... 28
RESULTS..... 28
DISCUSSION..... 29
CONCLUSION.....30
RECOMMENDATION.....31
APPENDICES.....33
ACKNOWLEDGEMENT.....33
STUDENTS DECLARATION.....33
Certificate of Originality and Authenticity.....34

TASK 1

1.1 INTRODUCTION

This experiment demonstrates the design and implementation of a proof of concept Smart Access System that combines a Cytron CT UNO, an MPU6050 IMU sensor, and an MFRC522 RFID module. The main objective is to establish a dual factor security mechanism for physical access control, requiring both an authorized RFID tag and a specific circular hand motion to unlock access. Once both authentication conditions are verified, the system triggers a servo motor to unlock and provides immediate visual feedback through LEDs.

The system integrates essential hardware components and applies key mechatronic principles. The MPU6050 IMU generates real time three axis accelerometer data, which is crucial for detecting and tracking the required hand motion using the I²C communication protocol. At the same time, the MFRC522 RFID reader uses the SPI protocol to read the unique identification (UID) from RFID tags, serving as the first layer of authentication. The Cytron CT UNO communicates with a Python script on a host computer via serial communication. This script processes both the motion and RFID data, performs gesture analysis, and verifies the UID before sending a command either ‘A’ (Access Granted) or ‘D’ (Access Denied) to control the servo motor and LED indicators.

The system is expected to effectively meet the dual factor verification objectives. The Python script should accurately distinguish the intended circular hand motion from other gestures using real time accelerometer XY data. As a result, access will be granted (servo unlocks and green LED turns on) only when both authentication factors are valid, while any failure in either factor will result in access denial (servo locks and red LED lights up).

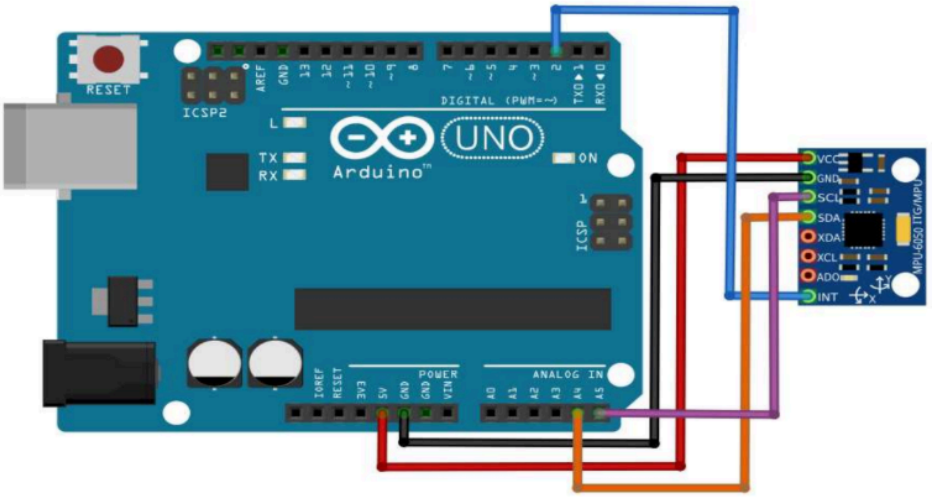
2.0 EXPERIMENT 1

2.1 Materials And Equipments

Materials and components that were used in the experiment:

- Cytron CT Uno
- MPU6050 sensor
- Jumper wire/breadboard
- USB cable
- Computer with arduino IDE and python installed

2.2 Experimental Setup



- Microcontroller: The Arduino Uno board was used to interface with the sensor and manage serial communication.
- Sensor: The MPU6050 sensor module was used to capture accelerometer data.
- Connectivity: Jumper wires and the breadboard were used for making electrical connections.
- Host System: The computer with the USB cable was required for power, programming,
and establishing the serial link.
- Software: The Arduino IDE was installed for programming the microcontroller, and Python with the serial and matplotlib libraries was installed for data processing and visualization.

2.3.1. Hardware Assembly

1. The VCC pin of the MPU6050 module was connected to the 5V power supply pin on the Cytron CT UNO.

2. The GND pin of the MPU6050 module was connected to the GND pin on the Cytron CT UNO.

3. The SDA (Serial Data) pin of the MPU6050 module was connected to the A4 (Analog 4) pin on the Cytron CT UNO.

4. The SCL (Serial Clock) pin of the MPU6050 module was connected to the A5 (Analog 5) pin on the Cytron CT UNO.

2.3.2 Software And programming

1. **Set Up COM Port Communication**

   The Cytron CT UNO is connected to the computer through a COM port, which enables serial communication between the board and the Python program. The correct port, **COM7**, is selected in both the Arduino IDE and the Python script.

2. **Initialize Serial Communication**

   The Cytron CT UNO begins communication using Serial.begin(9600) to establish data transfer between the microcontroller and the Python application.

3. **Initialize the MPU6050 Sensor**

   The MPU6050 sensor is connected to the Cytron CT UNO through the I²C pins (SDA → 20, SCL → 21) and activated using mpu.initialize() to measure acceleration and rotation.

4. **Continuously Read Sensor Data**

   The Cytron CT UNO continuously reads acceleration values (ax, ay, az) using mpu.getMotion6() to monitor real-time motion data.

5. **Transmit Data to Python via Serial**

   The measured sensor values are formatted as text (for example, 123,450,-980) and transmitted to the computer through the serial connection.

6. **Receive and Process Data in Python**

   The Python program reads the incoming data line by line, cleans it, and extracts the acceleration values for each axis.

7. **Store and Analyze Motion Data**

   The processed data is stored in buffers or arrays, then analyzed mathematically to identify circular motion patterns.

8. **Display the Results**

   The detected motion or gesture is shown in the Python terminal or visualized on a live graph using **Matplotlib**.

Code Used :

Arduino :

```
#include <Wire.h>

#include <MPU6050.h>


MPU6050 mpu;


void setup() {

  Serial.begin(9600);

  Wire.begin();

  mpu.initialize();

  if (!mpu.testConnection()) {

    Serial.println("MPU6050 connection failed!");

    while (1);

  }

}


void loop() {


  int ax, ay, az, gx, gy, gz;

  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  Serial.print(ax); Serial.print(",");

  Serial.print(ay); Serial.print(","); Serial.println(az);
```

```
delay(100);

}
```

Python :

```
import matplotlib

matplotlib.use('TkAgg')


import serial

import matplotlib.pyplot as plt




ser = serial.Serial('COM5', 9600, timeout=1)




plt.ion()

fig, ax = plt.subplots()

line, = ax.plot([], [], 'ro-', markersize=8)

ax.set_xlabel("ax")

ax.set_ylabel("ay")

ax.set_title("MPU6050 Live Data (Last 10 Points)")

ax.grid(True)


x_vals, y_vals = [], []


print("Streaming data... (Press Ctrl+C to stop)")
```

```
try:
    while True:
        data = ser.readline().decode(errors='ignore').strip()
        if not data:
            continue

        print(data)
        values = data.split(',')
        if len(values) >= 2:
            try:
                ax_val = int(values[0])
                ay_val = int(values[1])
            except ValueError:
                continue

            x_vals.append(ax_val)
            y_vals.append(ay_val)


            x_vals = x_vals[-10:]
            y_vals = y_vals[-10:]


            line.set_data(x_vals, y_vals)
            ax.relim()
            ax.autoscale_view()

            fig.canvas.draw_idle()
```

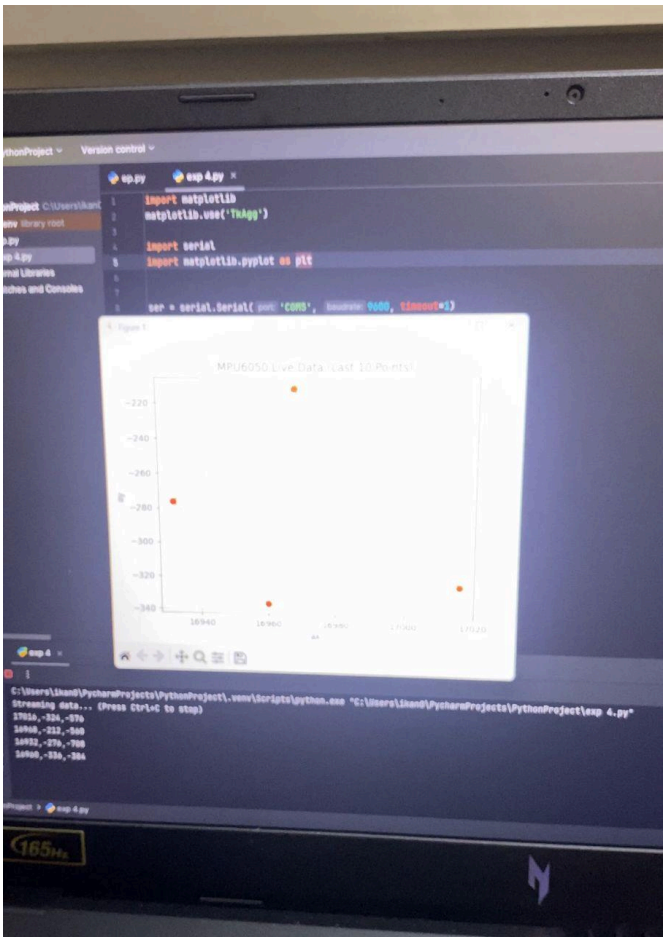
```
            plt.pause(0.5)


except KeyboardInterrupt:

    print("\nStopped by user.")

finally:

    ser.close()
```

IMPROVEMENT

1. Added matplotlib.use('TkAgg') - Fixes refresh issue inside PyCharm / Windows
2. Limited to last 10 readings - Prevents excessive clutter and keeps the most recent motion visible

3.0 RESULT

The successful execution of the experiment demonstrates that real-time data transmission from the Arduino to Python is achievable. The plotted graph accurately represents circular motion being ploΣed., validating the effectiveness of the serial communication process.



4.0 EXPERIMENT 2

4.1 Materials And Equipments

1. Cytron CT Uno
2. RFID reader (MFRC522) +RFID tags/cards
3. Servo motor
4. Red and green LEDs + resistors
5. MPU6050 sensors
6. Jumper wires
7. Breadboard
8. USB cables
9. RS232 to USB adapter cable

4.2 Experimental Setup

- The RS232 female connector was connected to the laptop using a USB to RS232 adapter (male pin to USB).

- The COM port of the RFID reader was verified through the Arduino Serial Monitor or the Device Manager.

- The RFID reader's USB connection was attached to the laptop to provide power.

- The entire circuit was then assembled.

- The Cytron CT UNO board was connected to the computer via USB for programming and serial communication.

- The Arduino code was uploaded, and Python was used to run the serial interface for UID detection, LED control and servo motor operation

4.3 Methodology

4.3.1 Circuit Assembly

1. **RFID Reader Connections**

- The USB desktop RFID reader was connected directly to the computer via the USB port and assigned as USB HID keyboard device.

- Unlike SPI-based RFID modules such as the MFRC522, the USB RFID reader does not connect to the Arduino pins. Instead, it operates as a keyboard-mode HID device, sending the card UID directly to the computer. The Python program receives the UID as if it were typed from a keyboard input.

2. Servo and LED Connections (Digital I/O)

The control devices were connected to the following digital output pins:

- The servo signal wire was connected to Digital Pin 9 (D9).

- The green LED (access granted indicator) was connected to Digital Pin 3 (D3).

- The red LED (access denied indicator) was connected to Digital Pin 4 (D4).

- The MPU6050 sensor was connected using SDA → A4and SCL → A5.

4.3.2. Integration explanation

1. **COM Port Configuration**

   The system operates using a single COM port for Arduino communication.

   - The RFID reader functions as a USB keyboard-mode HID device, so it does not require or appear as a COM port..
   - COM5 is connected to the Arduino, which receives control commands from the Python program to manage the servo motor and LED indicators.

2. **Arduino Initialization**

   The Arduino initializes the connected components at startup. The servo motor is attached to pin 9, while the green LED and red LED are connected to pins 3 and 4, respectively. The system begins in its default locked state, with the red LED turned on and the green LED turned off.

3. **RFID Data Acquisition via Python**

   The Python program continuously listens for input from the USB RFID reader, which operates in keyboard emulation (HID) mode. When a card is scanned, its UID is automatically typed into the program's input stream and captured by Python. The UID is then compared against a predefined list of authorized UIDs stored within the program

4. **Card UID Verification**

   If the scanned UID matches an authorized entry, Python identifies it as valid and transmits the character 'A' (Access Granted) to the Arduino. If the UID is not found in the list, Python sends 'D' (Access Denied).

5. **Command Execution on Arduino**

   Upon receiving the command through COM5, the Arduino performs different actions based on the input. For 'A' (Access Granted), it activates the green LED, performs a short delay, detects circular motion, and rotates the servo

motor to 180 degrees (unlock position). After several seconds, it returns the servo to 90 degrees (locked position) and restores the LEDs to their initial states. For ‘D’ (Access Denied), the Arduino flashes the red LED several times to indicate that access has been denied.

6. **System Standby and Reset**

   After executing the operation, the Arduino returns to standby mode, ready for the next RFID scan. The Python program remains active, continuously monitoring the RFID reader for new input.

4.3.3 Code Used

Arduino Code

```
#include <Wire.h>

#include <MPU6050.h>

#include <Servo.h>


MPU6050 mpu;

Servo servo;


// --- Pin definitions ---0013070376

0013070376

0013070376


#define GREEN_LED 4

#define RED_LED 3


// --- Variables ---

int servoPosition = 90;  // default: locked


void setup() {

  Serial.begin(9600);

  Wire.begin();


  // Initialize MPU6050
```

```
  mpu.initialize();
  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  // Setup servo & LEDs
  servo.attach(9);
  servo.write(servoPosition);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);  // start locked

  Serial.println("Arduino ready.");
}

void loop() {
  // --- 1 Read motion data and send to Python ---
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  Serial.print(ax);
```

```
  Serial.print(",");
  Serial.print(ay);
  Serial.print(",");
  Serial.println(az);
  delay(100);  // sampling rate ~10 Hz

  //
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    if (cmd == 'A') {

      servo.write(180);
      digitalWrite(GREEN_LED, HIGH);
      digitalWrite(RED_LED, LOW);
      Serial.println("Unlocked (A)");
    }
    else if (cmd == 'D') {

      servo.write(90);
      digitalWrite(GREEN_LED, LOW);
      digitalWrite(RED_LED, HIGH);
      Serial.println("Locked (D)");
    }
  }
```

```
}
```

Python Code

```
import serial

import keyboard

import time

import matplotlib.pyplot as plt




ARDUINO_PORT = 'COM5'

BAUD_RATE = 9600

AUTHORIZED_TAGS = ["0013070376"]




arduino = serial.Serial(ARDUINO_PORT, BAUD_RATE, timeout=1)




plt.ion()

fig, ax = plt.subplots()

ax.set_xlim(-20000, 20000)

ax.set_ylim(-20000, 20000)

line, = ax.plot([], [], 'ro-')



print("System ready. Scan RFID card...")


buffer = ""

motion_detected = False
```

```
try:
    while True:

        event = keyboard.read_event(suppress=False)
        if event.event_type == keyboard.KEY_DOWN:
            key = event.name


            if key.isdigit():
                buffer += key


            elif key in ["enter", "return"]:
                tag = buffer.strip()
                buffer = ""

                print(f"\nScanned Tag: {tag}")


                if tag in AUTHORIZED_TAGS:
                    print("Authorized! Perform circular motion with
MPU6050...")
                    time.sleep(1)

                    x_vals, y_vals = [], []
                    motion_detected = False
                    start_time = time.time()
```

```
                    while time.time() - start_time < 5:
                        data =
arduino.readline().decode(errors='ignore').strip()
                        if data:
                            values = data.split(',')
                            if len(values) >= 2:
                                ax_val = int(values[0])
                                ay_val = int(values[1])
                                x_vals.append(ax_val)
                                y_vals.append(ay_val)



                                if len(x_vals) > 10:
                                    x_vals.pop(0)
                                    y_vals.pop(0)

                                line.set_data(x_vals, y_vals)
                                plt.draw()
                                plt.pause(0.05)



                    if len(x_vals) > 5:
                        x_range = max(x_vals) - min(x_vals)
                        y_range = max(y_vals) - min(y_vals)
                        if x_range > 2000 and y_range > 2000:
                            motion_detected = True
```

```
                if motion_detected:
                    arduino.write(b'A')
                    print("A")
                    print("WELCOME BACK SIR!!")
                else:
                    arduino.write(b'D')
                    print("D")

            else:
                print("Unauthorized card!")
                arduino.write(b'D')

            print("\nSystem ready. Scan RFID card ...")

except KeyboardInterrupt:
   print("\nProgram stopped by user.")
finally:
   arduino.close()
```

5.0 RESULT

Motion Detection Approach

The system uses an MPU6050 motion sensor to detect movement.
When an authorized RFID card is scanned, the Python program starts reading acceleration values (`ax`, `ay`, `az`) from the MPU6050 through the Arduino.

The program observes how much these values change over time.
If there are large changes in both the X and Y directions, it means the user has moved the sensor in a circular motion.
If the readings stay almost constant, it means no motion was detected.

When circular motion is detected, the program sends a signal `'A'` to the Arduino to unlock (servo turns and green LED turns on).
If no motion is detected, it sends `'D'` to lock (servo returns and red LED turns on).

List of authorized UID

1. **0013070376**

6.0 DISCUSSION

This experiment aimed to design and analyze two standalone embedded systems: a hand gesture recognition system using the MPU6050 sensor, and an RFID based access control system. Both systems were developed to demonstrate real time sensor data acquisition, classification and actuator control, which are fundamental aspects of modern human computer interaction and secure access technologies.

Limitations and Sources of Error:

Hand Gesture System:

1. The use of fixed threshold values limited adaptability to different users and varying motion intensities.
2. Sensor readings were affected by noise, drift, and external vibrations, which influenced accuracy.
3. Overlapping gestures or small unintended movements sometimes caused incorrect classification.
4. The 100 ms loop delay helped reduce fluctuations but introduced a minor response lag.
5. The absence of advanced filtering or sensor fusion techniques reduced the system's stability and detection precision.

RFID System:

1. The RFID reader occasionally failed to detect tags when the tag orientation or distance was not ideal.
2. Manual UID registration using JSON was inefficient and impractical for managing a large number of authorized users.
3. The lack of encryption made the system vulnerable to security breaches in real world applications.
4. Serial communication speed required careful control to avoid data loss or delays, especially during rapid or repeated scans.

7.0 CONCLUSION

This experiment successfully developed two embedded systems: one for hand gesture recognition using the MPU6050 sensor and another for RFID based access control. Both systems demonstrated real time sensor data processing and actuator control, which are important aspects of modern automation and security systems. The hand gesture system accurately detected motion data and responded to user gestures, though minor errors occurred due to sensor noise and fixed threshold values. The RFID system effectively verified user identities and activated a servo motor with LED indicators to represent an unlocking action. Using serial communication, both systems worked together seamlessly, showing strong interaction between hardware and software components. Although there were small limitations in sensitivity, noise and data handling, the systems overall performed well and met the project objectives. The findings from this experiment highlight the potential of embedded systems in enhancing automation, user interaction and security in real world applications.

8.0 RECOMMENDATION

To improve both system efficiency and scalability, several upgrades can be introduced, focusing on better data management and user interaction. One key enhancement would be the addition of real-time data recording within the Python program. This feature would enable continuous tracking of sensor and RFID information, allowing users to review, analyze and troubleshoot system behavior more effectively.

For the hand gesture recognition module based on the MPU6050 sensor, adopting a more sophisticated recognition method such as a machine learning algorithm like k-nearest neighbors or decision trees could significantly boost detection precision. This improvement would allow the system to recognize subtle or closely related hand motions with greater reliability than the existing threshold-based technique.

In terms of software structure, implementing a modular programming design for both Arduino and Python codes is strongly advised. Separating sections responsible for hardware communication, logic operations and user interface will make the entire setup more flexible, easier to maintain, and simpler to update in the future. Enhancing the RFID system with secure authentication features such as encrypted UID verification would further increase system reliability and make it more suitable for real-world security or access control applications.

Finally, incorporating a graphical user interface (GUI) into the Python application using frameworks like Tkinter or PyQt would make the system more user-friendly. A GUI would allow users to visualize real-time data, adjust servo settings, and monitor gestures or card scans conveniently through an interactive dashboard.

10.0 APPENDICES

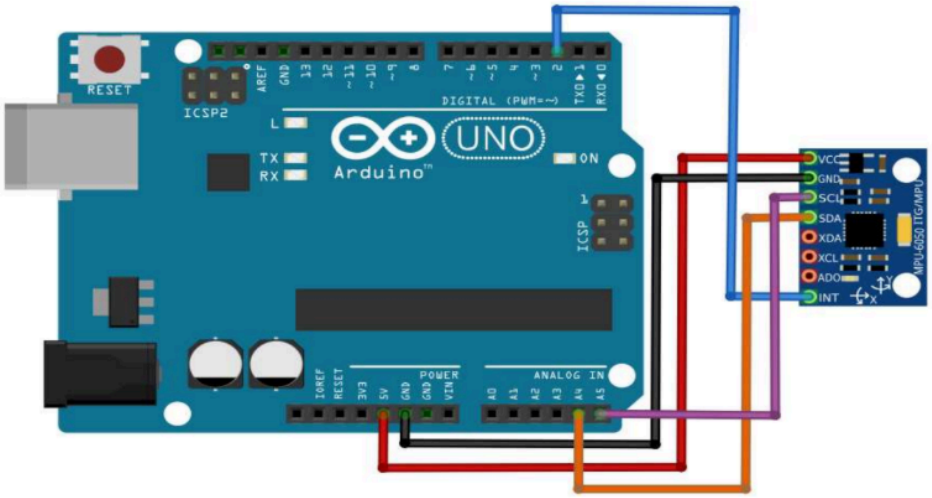


Figure 1

Figure 3

11.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help,encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

12.0 STUDENT'S DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any

plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: ***ahmadfarhan*** Read[/]
Understand[/]
Name: Ahmad Farhan bin Ahmad Nahrudi Agree[/]
Matric Card: 2317373

Signature: ***aimansafawi*** Read[/]
Understand[/]
Name: Muhammad Aiman bin Safawi Agree[/]
Matric Card: 2318249

Signature: ***nafizzuhairee*** Read[/]
Understand[/]
Name: Mohd Nafiz Zuhairee bin Mohd Neezar Agree[/]
Matric Card: 2317423